

A Generalizable Approach to Learning Optimizers

Diogo Almeida

Clemens Winter

Jie Tang

Wojciech Zaremba

OpenAI

Presenter: Huy Hoang Dang
DAC seminar WS22/23

Motivation

Introduction to LHOPT

Problem setup

1. Feature space
2. Action space
3. Reward Function
4. Inner Optimizer

Results

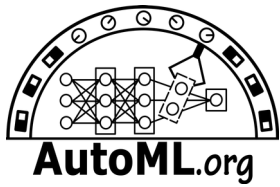
Limitations

Conclusion

References

Motivation

- Previous works of “learning to optimize” attempt to learn a general update rule **from scratch**.
 - Quite promising, even outperforming human-designed optimizers in specific tasks.
- But testing these results in **out-of-distribution**, they have shown limited generalization.
 - E.g: image-based datasets do not work in text-based datasets.
- Therefore, the proposed method manages to generalize to different tasks **without tuning**.



LHOPT – Learned Hyperparameters OPTimizers

A **generalization-first** perspective

Leverages the priors of existing optimization algorithms, such as **Adam**.

Can be interpreted as having **data-driven schedules** that **interpolate** with **hand-designed optimizers**.

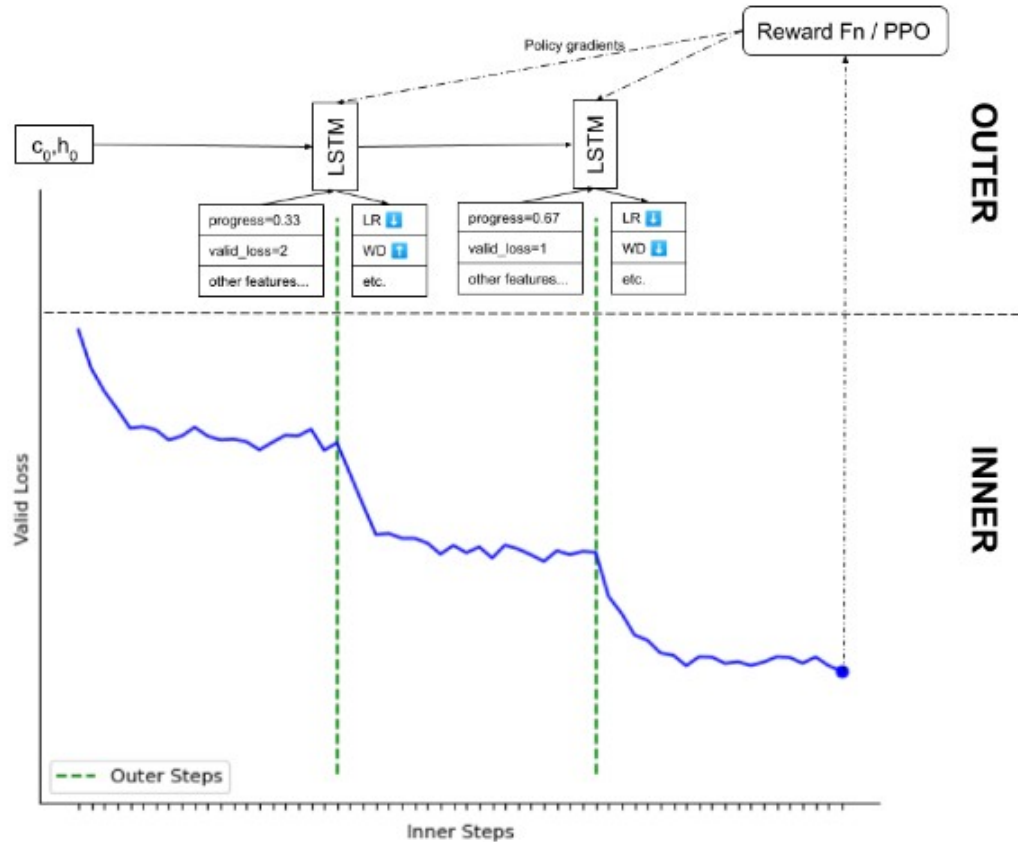
LHOPT – Learned Hyperparameters OPTimizers

Learning to **update hyperparameters** (e.g: learning rate, weight decay, etc.) instead of parameters.

This affords two main axes of **decoupling**:

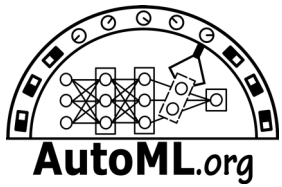
- 0 Network architecture
- 0 Frequency at which the policy has to run

Problem setup



Inner steps –
Applying the learned optimizer to an inner optimization task (optimizee).

Outer steps –
Collect input statistics from the inner steps (e.g validation loss) to update hyperparameters for the inner optimizer.



Problem setup (cont.)

Feature space

Action space

Reward function

Inner Optimizer

Most of the features are **unitless features** (e.g: log-ratio, cosine similarity, etc.)

Different from existing learned optimizers, which use only statistics such as the gradient, square of gradient, and loss values.

→ Prevent the policy from inferring with the inner optimization task.

CDF (Cumulative distribution function) feature is novel since:

- There are use cases for which we would like to know the **relative values** of a feature within an inner task.
- Ranking features should be **invariant** to details of the inner task.

Action space

Two ways of updating hyperparameters:

- Scaling (multiplying by a constant)
- Logit shifting

→ Making the model more robust by adding **relative actions** and **randomness**

Action space (cont.)

Additional class of actions – learning to restart from checkpoints

Three actions per checkpoint:

- Save current state
- Load state
- Swap states
- Do nothing

Ideas:

- Learned optimizer should be able to undertake **task-specific tuning**.
- Can be learned with **Population Based Training (PBT)**.
- Help with exploration while training by
 - Decreasing the length of credit assignment paths
 - Less punishing to sample suboptimal settings.

Reward function

Evaluate a baseline on the same problem with all task-specific values and initial weights.

Take all points on the **learning curve** of the baseline then fits a **power law curve**. This results in a function where:

- **0** corresponds to the worst loss
- **1** to the best loss
- **values greater than 1** take into account how difficult that problem

Designed **Customizable Inner Adaptive Optimizer (CIAO)**⁽¹⁾ to work with **LHOPT** formulation

- **Existing optimizers** have been designed to work with mostly **static** hyperparameters and **minimal** hyperparameter tuning.
- Easily create **new optimization techniques** as well as **ease of tuning**.

(1) <https://github.com/openai/LHOPT>

Basic Tasks

Table 6: Inner Task Distribution.

	Ratio
Algo + RNN	0.27
MNISTs + CNN	0.068
MNISTs + MLP	0.2
NQM	0.014
Text + RNN	0.14
Text + Transformer	0.31

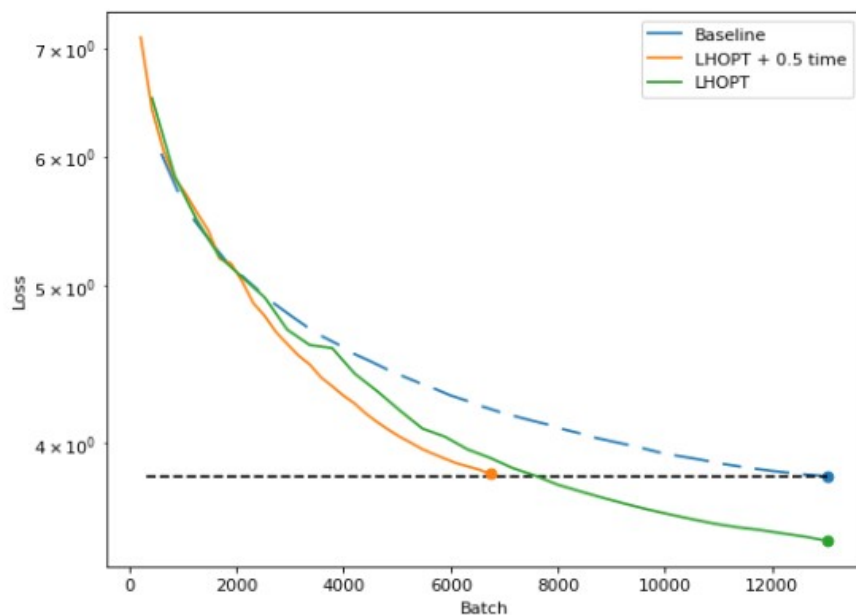
Table 1: Fraction of test tasks where the model gets a $\geq 2x$ speedup

	Default	OnlyMNISTs	OnlyNQM	Large
Overall	0.691	0.631	0.294	0.725
MNISTs + MLP	0.443	0.479	0.073	0.457
MNISTs + CNN	0.777	0.829	0.095	0.811
Algo + RNN	0.817	0.637	0.469	0.893
Text + RNN	0.827	0.719	0.478	0.853
Algo + Transformer	0.572	0.580	0.340	0.599
Text + Transformer	0.713	0.543	0.309	0.738

Main metric used: the fraction of test tasks where the model gets a $\geq 2x$ speedup.

The models show good generalization to **Algo + Transformer** tasks despite never being trained on them.

GPT-2 on WikiText-103



Baseline (AdamW) starts off better than **LHOPT** but then underperforms

Table 2: Language modeling performance on WikiText-103

Model	Test Perplexity
GPT2-Large	45.6
+ LHOPT + 0.5 time	46.1
+ LHOPT	32.5
+ LHOPT + OM + 0.5 time	63.4
+ LHOPT + OM	40.9

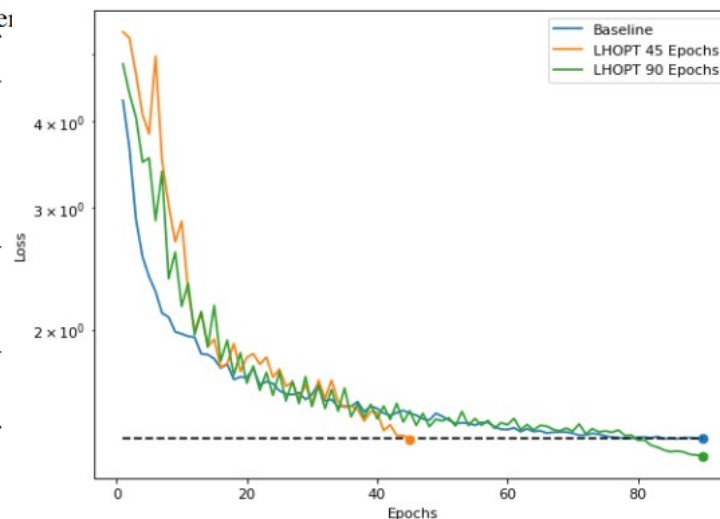
Results (cont.)

ResNet on ImageNet

Table 3: ImageNet performance of adaptive optimizers + SGD baselines for reference

Model	Epochs	Acc@1	Acc@5	Test Loss
Resnet18 + AdamW	90	67.32	87.52	1.39
Resnet18 + LHOPT	45	67.24	87.54	1.39
Resnet18 + LHOPT	90	68.89	88.43	1.31
Resnet18 + LHOPT + OM	45	66.54	87.09	1.41
Resnet18 + LHOPT + OM	90	68.34	88.54	1.31
Resnet50 + AdamW	90	71.42	89.66	1.35
Resnet50 + LHOPT	45	72.88	91.17	1.14
Resnet50 + LHOPT	90	73.52	91.38	1.07
Resnet18 + SGD	90	69.10	88.62	1.26
Resnet50 + SGD	90	74.66	92.09	1.01

LHOPT beats the baseline (**AdamW**) but slightly underperforms **SGD**



Results on Out-of-Distribution dataset

Neural Collaborative Filtering

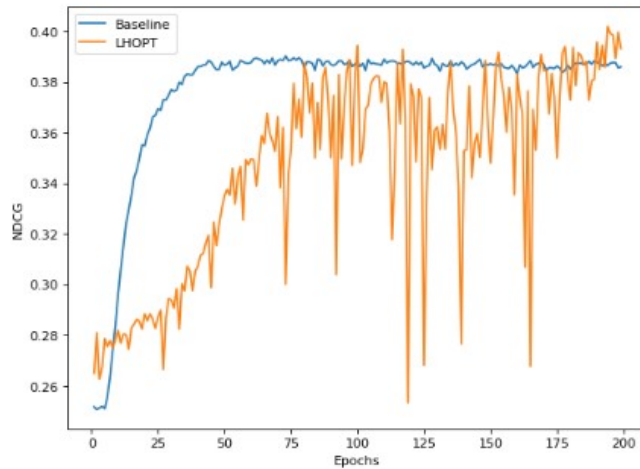
Table 4: NCF Performance. Larger is better for both metrics.

Model	NCDG	Hit Ratio
GMF	0.3677	0.6397
GMF + LHOPT	0.3134	0.5553
NeuMF	0.3859	0.6584
NeuMF + LHOPT	0.3932	0.6705

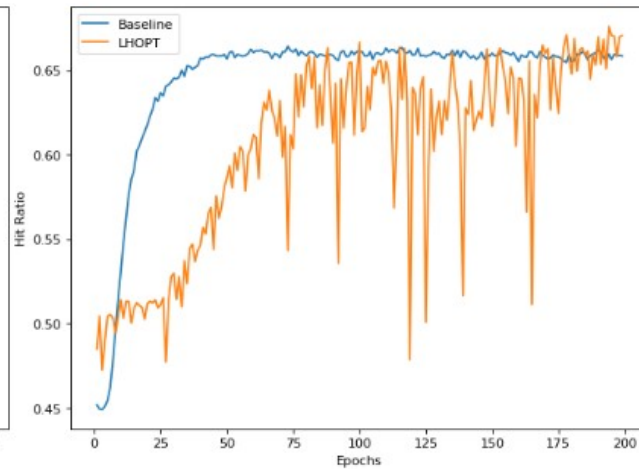
LHOPT underperformed the baseline for the **GMF models** but beat the baseline for the **NeuMF model**.

Results on Out-of-Distribution dataset (cont.)

Neural Collaborative Filtering



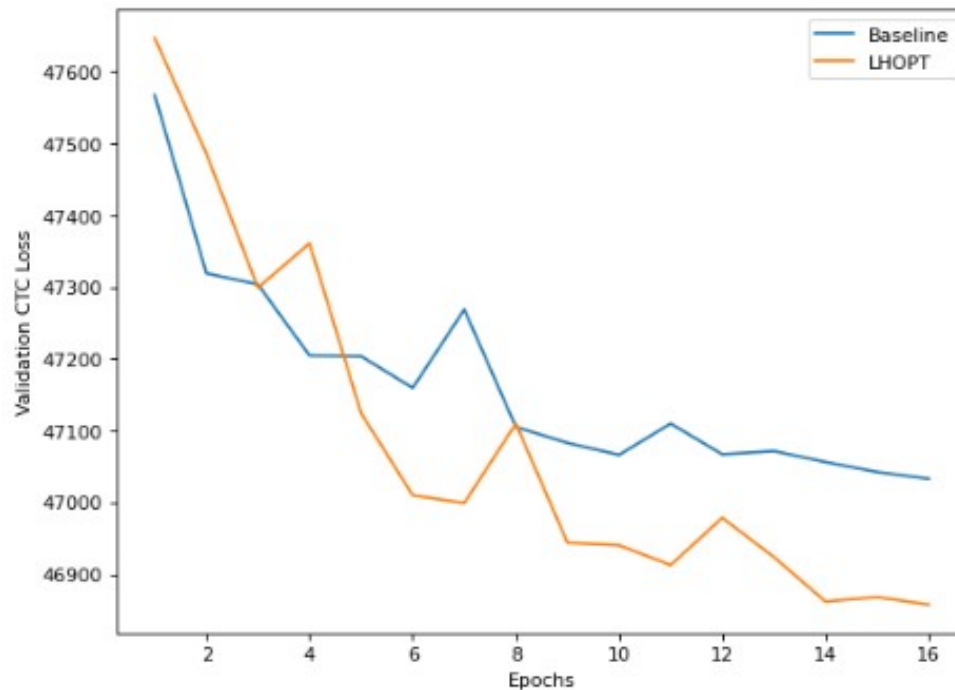
(a)



(b)

Results on Out-of-Distribution dataset (cont.)

Speech Recognition



Appendix – Initial Inner Hyperparameters

Table 7: Initial Inner Hyperparameters.

Hyperparameter	Value
Learning rate	$1e^{-3}$
$1 - \beta_1$	0.1
$1 - \beta_2$	$1e^{-2}$
ϵ	$1e^{-6}$
Weight decay	$1e^{-2}$
Grad clip fraction	0.8
$1 - \beta_{gradclip}$	$1e^{-2}$
Denominator	Adamax
Use LAMB	True
LAMB minimum trust [31]	$1e^{-3}$
$1 - \beta_{LAMB}$	0.05

Appendix (cont.) – Action & Noise

Table 8: Actions and Noise.

Hyperparameter	Action Type	Action Values	Noise
Learning rate	Scale	[0.5, 0.707, 0.9, 1, 1.1, 1.414, 2.0]	$[1e^{-2}, 1e^2]$
Weight decay	Scale	[0.5, 2.0]	[0.1, 10]
ϵ	Scale	[0.5, 2.0]	[0.1, 10]
$1 - \beta_1$	Scale	[0.5, 2.0]	[0.1, 10]
$1 - \beta_2$	Scale	[0.5, 2.0]	[0.1, 10]
Grad clip fraction	Logit Shift	[-1, -0.3, 0.3, 1]	[-1, 1]
$1 - \beta_{gradclip}$	Scale	[0.5, 1.0, 2.0]	[0.5, 2]
$1 - \beta_{LAMB}$	Scale	$[\frac{1}{1.5}, 1.0, 1.5]$	[0.5, 2]

Appendix (cont.) – List of features used in LHOPTs

π **Features** (features are added to the **policy network** π)

Features updated at each outer step:

- Progress [0, 1]
- Previous action representation
- Log loss ratio
- Log train/ valid loss
- ...

Features updated every 4 inner steps:

- Fraction of gradients clipped
- Cosine similarity of gradient and update
- Log of noise scale
- ...
- Other CDF features

Appendix (cont.) – List of features used in LHOPTs

V Features (features are added to the **value function**)

- A vector representation of the task
- A vector representation of the random initial hyperparameters
- Reward
- Log train and eval loss
- Statistics about the reward baseline optimizer's losses

Limitations

1. This metric ignores how much better or worse the models are than baselines.
2. Since training loss and validation loss are also used as features, overfitting to the validation set might occur.
3. A single run cannot provide an exact measurement of a speedup; it can only provide an approximate upper or lower bound.
4. ...

Conclusion

The **generalization-first** approach used in this study makes **LHOPTs** more useful and effective.

- Due to high computing costs, inability to be generalized, etc., previous efforts train update rules from scratch, which is not really feasible in the real world.

My opinion:

- The performance and stability can be enhanced because of overfitting problems and the metric's limitations.
- This approach is relatively new, so really looking forward to improvements.

